DrillBit

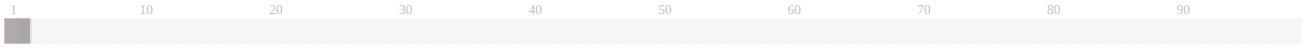
The Report is Generated by DrillBit Plagiarism Detection Software

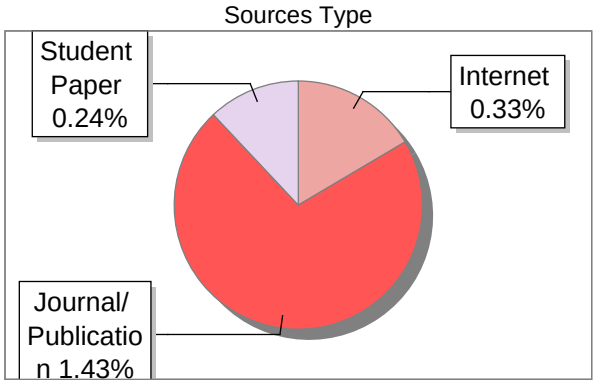
Submission Information

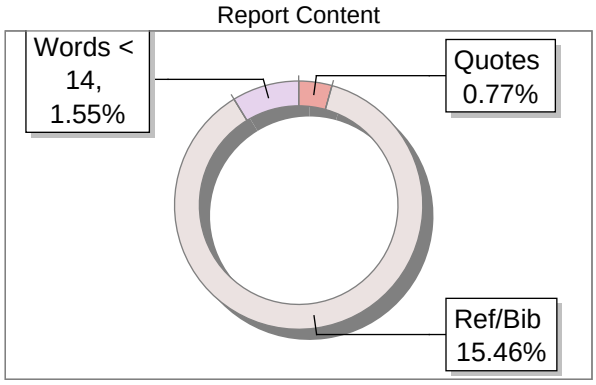
Author Name	MOHITH RAJ K KOTIAN
Title	CyberGuard: Intelligent Cybercrime Complaint Management System
Paper/Submission ID	7625236
Submitted by	akshitha.office@sode-edu.in
Submission Date	2026-11-13 15:18:30
Total Pages, Total Words	49, 13192
Document type	Project Report

Result Information

Similarity **2 %**







Exclude Information

Quotes	Excluded
References/Bibliography	Excluded
Source: Excluded < 14 Words	Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

DrillBit

DrillBit Similarity Report

2	6	A
SIMILARITY %	MATCHED SOURCES	GRADE

A-Satisfactory (0-10%)
B-Upgrade (11-40%)
C-Poor (41-60%)
D-Unacceptable (61-100%)

LOCATION	MATCHED DOMAIN	%	SOURCE TYPE
1	pdfcoffee.com	<1	Internet Data
2	Thesis Submitted to Shodhganga Repository	<1	Publication
3	118.151.209.1338080	<1	Publication
4	REPOSITORY - Submitted to VEMANA INSTITUTE OF TECHNOLOGY on 2025-12-22 09-43 5074971	<1	Student Paper
5	gnits.ac.in	<1	Publication
6	arxiv.org	<1	Publication

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi -590014



A PROJECT REPORT ON

CyberGuard: Intelligent Cybercrime Complaint Management System

In partial fulfillment of requirements for the degree of

Bachelor of Engineering

In

Computer Science & Engineering

SUBMITTED BY:

Mohith Raj k Kotian	**4MW23CS076**
Nikhil G Shetty	**4MW23CS082**
Pradyumna Shettigar	**4MW23CS093**
Pruthvi Shetty	**4MW23CS108**

Under the Guidance of

Dr. Sadananda L

Associate Professor & Head
Dept. of CSE



Department of Computer Science and Engineering

Shri Madhwa Vadiraja Institute of Technology & Management

Vishwothama Nagar, Bantakal -574 115

2026-2027

SHRI MADHWA VADIRAJA INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(A Unit of Shri Sode Vadiraja Mutt Education Trust ®, Udupi)

Vishwothama Nagar, BANTAKAL – 574 115, Udupi District, Karnataka, INDIA

Department of Computer Science & Engineering



CERTIFICATE

Certified that the Project Work titled **"CyberGuard: Intelligent Cybercrime Complaint Management System"** is carried out by **Mohith Raj K Kotian** (4MW23CS076), **Nikhil G Shetty** (4MW23CS082), **Pradyumna Shettigar** (4MW23CS093), and **Pruthvi Shetty** (4MW23CS108), bonafide students of **Shri Madhwa Vadiraja Institute of Technology and Management**, in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science & Engineering of Visvesvaraya Technological University, Belagavi, during the academic year 2026–2027. It is certified that all the corrections and suggestions indicated during the Internal Assessment have been incorporated in the report. The report has been approved as it satisfies the academic requirements in respect of Project Work prescribed for the said degree.

Dr. Sadananda L
Project Guide & Head
Dept of CSE
Signature with date

Dr. Nagaraj Bhat
Principal
SMVITM, Bantakal
Signature with date and seal

External Viva

Name of the Examiners: **Signature with Date**

1.

2.

DECLARATION

We declare that the project report titled "CyberGuard: Intelligent Cybercrime Complaint Management System", submitted towards the Bachelor of Engineering degree in Computer Science and Engineering at Shri Madhwa Vadiraja Institute of Technology and Management (SMVITM), Bantakal, Udupi, records our project work carried out under the supervision of Dr. Sadananda L.

This work has not been submitted for the award of another degree at this or any other institution. Sources used in the report are acknowledged through citations and the reference list.

Signature(s) of the Student(s):

Sl. No.	Name of the Student	Signature
1.	Mohith Raj K Kotian	
2.	Nikhil G Shetty	
3.	Pradyumna Shettigar	
4	Pruthvi Shetty	

ACKNOWLEDGEMENTS

We thank Shri Madhwa Vadiraja Institute of Technology and Management (SMVITM), Bantakal, Udupi, for the facilities and institutional support needed to develop and test CyberGuard.

We thank our Principal, Dr. Nagaraj Bhat, for supporting the project and its completion.

We acknowledge Dr. Sadananda L, Associate Professor and Head of the Department of Computer Science and Engineering, for the departmental support provided during this work.

Our project coordinators, Dr. Soumya J. Bhat, Ms. Rukmini Bhat B, and Ms. Rajeshwari, helped organise reviews and guided us through the project requirements. We thank them for their time and assistance.

We are particularly grateful to our guide, Dr. Sadananda L, for reviewing the work, questioning design choices, and helping us improve the implementation and report.

We thank the faculty and staff of the Department of Computer Science and Engineering for their assistance throughout the project.

Our parents, friends, and team members supported us through development, testing, and report preparation. We appreciate their patience and encouragement.

Mohith Raj K Kotian [4MW23CS076]

Nikhil Shetty [4MW23CS082]

Pradyumna Shettigar [4MW23CS093]

Pruthvi Shetty [4MW23CS108]

ABSTRACT

CyberGuard is a web and mobile platform for submitting, categorising, and tracking cybercrime complaints. We implemented a Node.js/Express API backed by MongoDB, with a separate Flask service for text classification. PrivacyShield replaces supported identifiers in the model-input narrative; it does not anonymise the entire database or uploaded evidence. The deployed classifier uses word-level TF-IDF with 1–2 n-grams and a maximum of 10,000 features, followed by Logistic Regression (C=2.0, balanced weights, lbfgs). A confidence gate at tau = 0.20 refers low-scoring inputs to the fallback and human review. Role-based access controls complaint operations, while Socket.IO carries status and message events. Evaluation used 1,950 balanced synthetic complaints across 13 classes (150/class), spanning 130 scenario families. The selected configuration obtained 79.48% ± 3.21% accuracy and Macro-F1 = 0.7507 in 5-fold GroupKFold cross-validation. On the group-disjoint held-out test (N=390), accuracy was 88.97% (347 / 390 correct), with Macro-F1 = 0.8896. A separate challenge set contained 215 items (195 in-domain + 20 benign queries); its reported in-domain raw and gated accuracies were 98.46% and 97.44%, respectively. These synthetic-data results support prototype evaluation, not a claim of operational readiness. Classification, escalation, and OCR outputs remain subject to authorised human review.

TABLE OF CONTENTS

CONTENTS	PAGE NO.
Acknowledgements	i
Abstract	ii
Table of Contents	iii-iv
List of Figures	v
List of Tables	vi
List of Acronyms and Abbreviations	vii
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope of the Project	2
1.5 Proposed Solution / Overview	3
1.6 Methodology	3
CHAPTER 2: LITERATURE REVIEW	5
2.1 Introduction	5
2.2 Review of Existing Systems	5
2.3 Comparison of Existing Systems	7
2.4 Research / Project Gap	7
2.5 Summary	8
CHAPTER 3: REQUIREMENTS AND SYSTEM ANALYSIS	9
3.1 Existing System	9
3.2 Limitations of Existing System	9
3.3 Proposed System	9
3.4 Functional Requirements	10
3.5 Non-Functional Requirements	10
3.6 Hardware Requirements	11
3.7 Software Requirements	11

CHAPTER 4: SYSTEM DESIGN AND METHODOLOGY 13

4.1 System Architecture 13

4.2 System Workflow / Block Diagram 13

4.3 Module Description 14

4.4 Data Flow / Process Flow 16

4.5 Database Design (if applicable) 16

4.6 Algorithm / Model / Technique Used 17

4.7 Methodology 18

CHAPTER 5: IMPLEMENTATION 20

5.1 Development Environment 20

5.2 Implementation Details 21

5.3 Module-wise Implementation 22

5.4 Algorithms / Models Implementation 23

5.5 User Interface 24

5.6 Application User Interface and Implementation Screens 25

CHAPTER 6: RESULTS AND DISCUSSION 28

6.1 Experimental Setup / Test Cases 28

6.2 Results 28

6.3 Performance Evaluation 29

6.4 Comparison with Existing Methods 31

6.5 Discussion 32

6.6 Summary of Result 32

CHAPTER 7: CONCLUSION AND FUTURE SCOPE 33

7.1 Conclusion 33

7.2 Limitations 33

7.3 Future Scope 34

REFERENCES 36

RESEARCH PUBLICATION **38**

List of Figures

Figures ..... **Page No.**
Figure 4.1 System Architecture of CyberGuard 13
Figure 4.2 Activity Diagram for Cybercrime Complaint Processing 14
Figure 4.3 PrivacyShield Model-Input PII Masking Pipeline 15
Figure 4.4 Methodology of CyberGuard 19
Figure 5.1 Citizen Complaint Submission Portal 25
Figure 5.2 Citizen Incident Tracking and Custody Timeline 25
Figure 5.3 Investigating Officer Command Matrix 26
Figure 5.4 Case Forensic Record and Dispute Protocols 26
Figure 5.5 Administrative Threat Radar and Velocity Analytics 27
Figure 5.6 Statistical NLP Classification Engine Telemetry and Playground 27
Figure 6.1 Class Distribution of the 1,950-Narrative Corpus across 13 Classes 29
Figure 6.2 Confusion Matrix of the Baseline Model on the 390-Sample Held-Out Test Partition 31
Figure 6.3 Candidate Classifier Comparison (5-fold GroupKFold cross-validation) 30

List of Tables

Tables Page No.

Table 2.1: Comparison of Existing Complaint Management Approaches.....7
Table 3.1: Hardware Requirements.....11
Table 3.2: Software Requirements 12
Table III: Reconciliation of Document and Repository Claims.....10
Table 5.1: Technologies Used in CyberGuard 20
Table II: System Technology Stack and Service Topology 20
Table I: Implemented Components and Feature Boundaries 21
Table 6.1: Functional Test Cases.....28
Table IV: Candidate Classifiers and Evaluation Metrics (5-fold GroupKFold cross-validation).....30
Table 6.2: Comparison of Complaint Management Approaches 31
Table V: Limitations and Required Validation 34

List of Acronyms and Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
DB	Database
JWT	JSON Web Token
ML	Machine Learning
NB	Multinomial Naive Bayes
NLP	Natural Language Processing
OCR	Optical Character Recognition
PII	Personally Identifiable Information
RBAC	Role-Based Access Control
REST	Representational State Transfer
SVM	Support Vector Machine
TF-IDF	Term Frequency–Inverse Document Frequency
UI	User Interface
URL	Uniform Resource Locator
OTP	One-Time Password
WS	WebSocket

CHAPTER 1
INTRODUCTION

1.1 Background

Cybercrime reporting requires more than a submission form. A complainant may describe a UPI transfer, an impersonated social-media account, a phishing message, or sustained harassment in the same free-text field. The receiving desk must identify the incident type, associate evidence with the record, and decide who should review it. CyberGuard addresses this processing stage rather than treating online submission as the complete solution.

In the manual workflow considered here, personnel read each narrative before assigning a category and routing the complaint. A basic digital form changes how the narrative arrives but need not change that work. Categorisation, attachment handling, and follow-up can remain separate tasks, with inconsistent labels and duplicated effort between desks.

We built CyberGuard around a shared complaint record. Web and mobile clients submit incident details to the backend; a classification service suggests a category, and staff use role-specific views to review the result. PrivacyShield masks supported identifiers in text sent for inference. The stored raw narrative remains sensitive and requires access controls.

Two design decisions shape the workflow. First, TF-IDF and Logistic Regression convert a free-text description into a category suggestion. Second, server-side role checks restrict access to complaint operations and fields. Masking reduces identifier exposure at the model boundary; authorisation controls who can retrieve the underlying record.

UPI scams, fake customer-care contacts, phishing links, and social-media impersonation provide representative reporting scenarios for this project. Public reporting channels already exist, including the National Cyber Crime Reporting Portal [13]. CyberGuard does not replace that infrastructure. Its engineering objective is to connect intake, category prediction, privacy processing, and case tracking within a single prototype workflow.

Complaint length varies substantially. A short account-takeover report may contain only a few useful terms; a harassment report may include dates, repeated incidents, and several parties. Both require careful review. The classifier supplies an initial label so that routine sorting does not consume all the attention available for ambiguous cases.

Routing also exposes data. Phone numbers, e-mail addresses, account details, and UPI identifiers can appear inside the narrative rather than in dedicated fields. Sending that text unchanged to every component creates unnecessary copies. Privacy-aware reporting research motivates limiting such exposure [1], [2]; in CyberGuard, supported identifiers are replaced before model inference.

1.2 Problem Statement

The problem addressed by CyberGuard is the gap between receiving a cybercrime complaint and producing a reviewable case record. Complainants need acknowledgement and visible progress. Staff

need a category, relevant evidence, a status history, and a way to identify potentially urgent reports. These functions must operate without giving every user unrestricted access to personal information.

We organised the problem into three requirements. Unstructured narratives need an initial category before routing. Shared records need different permissions for complainants, investigators, and administrators. Safety-related reports need explicit escalation rules rather than relying solely on queue order. The application therefore combines category prediction, model-input masking, role-based access, and a recorded status workflow. Each component supports staff decisions; none establishes that an offence occurred or that a case has a particular legal status.

The report follows those requirements through design and evaluation. Client interfaces implement reporting, the Flask service supplies classification, PrivacyShield limits model-input exposure, and backend permissions govern access. Timeline records and Socket.IO events support follow-up without moving case discussions into unrelated channels.

1.3 Objectives

The implementation objectives are as follows:

- Provide a shared intake and management workflow for complaints submitted through web and mobile clients.
- Collect an incident narrative and supporting evidence through a guided submission form.
- Generate an initial category from complaint text using a supervised NLP classifier, with review of uncertain outputs.
- Mask supported personally identifiable information in the narrative before it reaches the classification service.
- Enforce role-specific access to complaint operations and sensitive fields on the backend.
- Expose complaint status and permitted messages to the complainant and responsible personnel.
- Flag configured safety-related categories for early review through explicit escalation rules.
- Check application behaviour through functional tests and measure classifier performance on separated evaluation data.

These objectives divide the implementation into application services and text-processing services. Submission, evidence references, status changes, and messages belong to the application. Classification, masking, and escalation contribute structured outputs to the same record. The Express backend coordinates both paths.

1.4 Scope of the Project

The project covers complaint submission, evidence attachment, initial categorisation, staff review, status updates, and closure tracking. A complainant can submit an incident and retrieve its progress. Authorised personnel can inspect the record and update it through the permitted workflow.

The clients communicate with a Node.js backend and MongoDB. PrivacyShield runs within the backend before the inference request, rather than as another public service. Backend handlers also apply access restrictions and escalation rules. Socket.IO delivers event notifications to connected users.

The scope is administrative support, not autonomous investigation. A predicted label is not a legal conclusion, an uploaded file is not verified evidence merely because it was stored, and a status change is not proof of action outside the application. Model behaviour is bounded by the training corpus and its category definitions.

The classifier uses thirteen categories covering financial fraud, phishing, identity theft, cyberbullying, hacking, ransomware, data breach, online fraud, child exploitation, women and child safety, network attacks, crypto fraud, and other. Government hosting, production police-register integration, and certification of evidentiary admissibility remain outside this version. Chapter 7 identifies the validation needed before wider use.

1.5 Proposed Solution / Overview

CyberGuard links complaint intake to a structured review workflow.

A complainant enters incident details in the web or mobile client and attaches available evidence. The backend validates the request, prepares a masked narrative, and calls the Flask classifier. Word-level TF-IDF features and Logistic Regression return a proposed category, subject to the confidence gate.

MongoDB holds the complaint, predicted category, priority, and related history. Authorised staff retrieve the record through the backend, change its status, and exchange case-linked messages. Configured escalation conditions flag safety-related cases at intake. The raw narrative remains in storage, so masking at the inference boundary must not be confused with database anonymisation.

The processing order is the main distinction from a store-only form: validate, mask the inference text, classify or invoke the fallback, evaluate escalation, and record the outcome. The client receives confirmation and a tracking reference. Reviewers can then begin with a structured record rather than an uncategorised paragraph.

1.6 Methodology

Development was organised around the following stages:

1. **Requirements analysis: define the tasks and access needs of complainants, administrators, and investigators.**
2. **System design: specify service boundaries, data entities, workflow states, and permission checks.**
3. **Client development: implement web and mobile forms, tracking views, and staff interfaces.**
4. **Backend development: implement authentication, complaint handling, evidence references, messages, and persistence.**
5. **Model development: compare TF-IDF representations and classifiers, then deploy the selected word-level Logistic Regression pipeline.**
6. **Privacy and authorisation: apply identifier masking before inference and enforce role restrictions when records are accessed.**
7. **Testing: exercise the main application paths and evaluate the classifier using group-separated data.**

8. **Analysis: interpret observed results, document unresolved risks, and identify the next validation steps.**

The stages informed an iterative implementation rather than a single handover between phases. Changes to masking affect the text seen by the classifier; changes to category mapping affect the queues shown to staff. Integration testing therefore checks these interfaces together, alongside the independent model evaluation reported in Chapter 6.

CHAPTER 2
LITERATURE REVIEW

2.1 Introduction

A complaint platform must collect an incident, organise it for review, control access to sensitive material, and retain a usable history. These are related but distinct tasks. This review considers reporting systems and language-processing research in terms of the design choices relevant to CyberGuard, rather than assuming that a digital form provides all four functions.

The literature offers several ways to connect those tasks. Language models can assist reporting and classification; a smaller supervised classifier can provide a narrower category suggestion. Privacy-aware preprocessing limits the identifiers passed to a model, while application-level permissions govern access to the case. These mechanisms address different risks and must be evaluated separately.

The review is organised around reporting interfaces, complaint classification, privacy controls, and tracking. For each area, the discussion separates the cited work from the engineering choices made in CyberGuard. Results obtained on another corpus are not treated as directly comparable to this project's synthetic-data evaluation.

2.2 Review of Existing Systems / Related Work

2.2.1 Digital Cybercrime Reporting Systems

Digital reporting replaces a physical submission step with a remote interface, but the functions behind that interface vary. Some platforms primarily collect reports; others incorporate communication or automated processing. The relevant design question is not whether submission is online, but what validated information reaches the receiving desk and what the complainant can subsequently track [1], [15].

Mahmoud, Shishah, and Mistry developed ReACT_OCRS, an anonymous, voice-based reporting architecture [1]. Their system uses LLAMA3 to coordinate reporting actions, with speech processing and complaint classification based on LSTM and BERT components. CyberGuard takes a narrower route: its Flask service predicts a category using TF-IDF and Logistic Regression. This reduces the scope of inference, but it does not reproduce the conversational or speech-reasoning capabilities of ReACT_OCRS.

Sakpere, Kayem, and Ndlovu examined secure mobile reporting through the CryHelp App in a university setting with constrained technology resources [15]. Their usability evaluation makes interface operation an explicit part of system assessment. For CyberGuard, the practical lesson is to evaluate whether users can complete the reporting task, not merely whether the backend accepts a request. Category prediction also remains distinct from legal interpretation; risk management and human oversight are still necessary [16].

These systems address different parts of intake. ReACT_OCRS explores AI-assisted interaction, while CryHelp examines secure, usable reporting in its deployment context. CyberGuard uses this distinction to keep two interfaces explicit: the citizen's reporting interface and the structured record presented to personnel after processing.

2.2.2 Machine Learning for Text Classification

Supervised complaint classification maps a narrative to a predefined label using examples of previously labelled text. Rani et al. studied this task for Hinglish complaints with language-adapted transformer models [2]. Their setting is particularly relevant because code-mixed descriptions cannot be assumed to behave like standard English input.

We selected a word-level TF-IDF and Logistic Regression baseline for CyberGuard's thirteen-label task. The representation is sparse, the model is small enough for a separate local service, and probability estimates are available for a rejection gate [4], [5]. These properties support the prototype's deployment needs. They do not establish superiority over transformer models or reliability on unfamiliar wording.

2.2.3 Natural Language Processing in Complaint Management

Rani et al. combine privacy-aware preprocessing, augmentation of under-represented categories, and fine-tuning of Hinglish-oriented models [2]. Their approach treats language variation and category imbalance as model-development issues rather than problems that a submission form can resolve. CyberGuard's balanced synthetic corpus removes unequal class counts from its controlled evaluation, but it does not represent the linguistic distribution of real complaints.

The useful output for this workflow is a category field linked to the narrative. Staff can filter and assign records using that field while retaining the original description for authorised review. This is a bounded NLP task: predicting a label does not demonstrate general comprehension of the incident or establish the truth of the allegation.

The transformer study provides evidence for language-specific modelling on its own dataset [2]. CyberGuard instead evaluates a classical baseline under scenario-group separation. The two evaluations answer different questions and should not be ranked by their headline accuracy values. Kamath et al. also apply multiple transformer models to dark-web threat classification [3], but that task and its source data differ from citizen complaint triage.

The dark-web study connects text ingestion, classification, and reporting through a multi-model architecture [3]. Its relevance here is architectural: classification becomes useful when its output is delivered to a review process. CyberGuard applies that principle with a smaller service boundary. OCR provides a separate supporting function by extracting text from evidence images; it does not validate the extracted statements or determine the complaint category by itself [10].

2.2.4 Privacy Protection and Access Control

Privacy controls must account for identifiers embedded in narrative text. ReACT_OCRS incorporates anonymisation mechanisms [1], and the Hinglish complaint study replaces personal entities during preprocessing [2]. CyberGuard uses regular-expression substitution for supported formats. A matched phone number or e-mail address becomes a token before inference; unusual formats, contextual identifiers, and uploaded file contents require additional controls.

Masking and authorisation are separate boundaries. Masking limits what the model receives; backend permissions limit which account can read or change a complaint. The database still retains the raw

narrative in this implementation. It would therefore be inaccurate to describe PrivacyShield as complete anonymisation or as a substitute for protecting stored records.

2.2.5 Complaint Tracking and Real-Time Communication

Tracking requires persistent state as well as notification. CyberGuard records status changes in the complaint history and uses Socket.IO to notify connected clients [19]. A notification reports that an application event occurred. The stored record, not the socket connection, supplies the history available on a later visit.

Case-linked messages also keep communication associated with the relevant complaint. This can support handover between authorised personnel without relying on separate personal-message threads. The project has not measured a reduction in follow-up calls or handling time, so those potential benefits are not presented as observed results.

2.3 Comparison of Existing Systems

Table 2.1 compares broad workflow approaches rather than named products. Manual handling relies on personnel for each stage; a basic portal adds digital intake; CyberGuard connects intake to classification, masking, access checks, and tracking. The comparison describes scope, not measured performance.

The table lists the functions considered in this project. Features of an actual external platform depend on its implementation and operating procedures; the "basic online reporting" column must not be read as a survey result for all deployed portals.

Table 2.1: Comparison of Existing Complaint Management Approaches

Feature	Manual Complaint Handling	Basic Online Reporting	Proposed CyberGuard System
Complaint submission	Paper or direct interaction	Online forms	Web and mobile interfaces
Complaint classification	Mainly manual	May require manual review	ML-based category prediction
Privacy masking	Manual procedures	Depends on implementation	Pattern-based PII masking
Access control	Organizational procedures	Depends on implementation	Role-Based Access Control
Complaint tracking	Manual follow-up	May provide status tracking	Complaint status management
Communication	Direct or separate channels	Varies by system	Real-time communication features

Within this simplified comparison, CyberGuard covers the selected functions in one prototype. This is an integration claim, not evidence that it is the only system with those capabilities. The cited reporting studies already combine several functions, and a specialised platform may implement an individual function more thoroughly [1], [15].

Each approach retains a different dependency. Manual triage depends on available reviewer time. A store-only portal still needs interpretation after submission. Automated triage depends on data coverage and error handling. CyberGuard combines these approaches by generating a preliminary label and retaining staff review for uncertain or consequential decisions.

2.4 Research / Project Gap

The project gap is a practical integration problem: transform an incoming narrative into a record that can be reviewed, routed, and tracked without sending unnecessary identifiers to the classifier. The literature demonstrates relevant components [1], [2], [15], but their data, language support, and operating settings do not by themselves validate CyberGuard's proposed workflow.

We therefore focus on the handover between components. The backend validates the request, constructs

masked inference text, obtains a category or fallback result, evaluates escalation, and stores the resulting metadata. Role checks then control record access. This defines a testable workflow without asserting that equivalent integration is absent from all previous systems.

CyberGuard's contribution is the implementation and evaluation of that workflow as an engineering prototype. The classifier, PrivacyShield, role checks, timeline, and event channel are assessed according to their assigned functions. Chapter 6 distinguishes observed application behaviour from statistical classification performance.

Two limits frame the design. A category prediction depends on the training distribution, and a masking pattern detects only formats it recognises. Neither mechanism is a guarantee. Human review and additional privacy validation remain necessary, especially where safety-related complaints or sensitive evidence are involved.

2.5 Summary

The reviewed work separates three concerns: how people submit reports, how text is categorised, and how sensitive records are handled. ReACT_OCRS and CryHelp illustrate different reporting designs [1], [15]; the Hinglish classification study addresses language and label-distribution issues [2]. CyberGuard uses these distinctions to define its own service boundaries.

The project does not attempt to reproduce a large language model's reporting dialogue or claim a general advance in text classification. It implements a constrained classifier alongside masking, permissions, and tracking. Its value must be assessed through the correctness of those connections and the limits of the evaluation data.

The main unresolved requirements concern representative data, incomplete identifier detection, access to raw stored text, and safe handling of model uncertainty. Chapter 3 translates these concerns into functional and non-functional requirements.

The design carries forward two principles. Classification must be evaluated on data separated from model development, and privacy controls must be specified at each point where sensitive text is transmitted or retrieved. Those principles lead to group-aware evaluation, a masked model-input path, and backend-enforced access restrictions.

CHAPTER 3
REQUIREMENTS AND SYSTEM ANALYSIS

3.1 Existing System

The baseline workflow considered in this analysis has two forms: a counter-based process and a digital intake form followed by manual review. An online form removes the need to submit the initial report in person, but the extent of later automation depends on the platform. We use the store-only portal as a comparison case, not as a description of every existing service.

After intake, staff still need to identify the category, associate attachments, record progress, and respond to the complainant. Where these tasks use disconnected records, the handler must reconcile information across channels. CyberGuard brings the corresponding application functions behind one backend.

Routine descriptions and ambiguous reports should not require identical processing effort. A preliminary category can support filtering of familiar cases, while low-confidence and safety-related inputs receive explicit review. The requirements therefore include both automated assistance and a route for handling its failures.

3.2 Limitations of Existing System

The comparison workflow exposes the following limitations:

Manual category assignment consumes reviewer time and may vary between handlers.

A store-only submission form does not produce a category from the narrative.

Unmasked descriptions can expose personal identifiers to components or users that do not need them.

Without a tracking view, the complainant must request progress through another channel.

Separate messaging channels can fragment the record used during handover.

A general queue does not itself identify or prioritise safety-related complaints.

Disconnected complaint and attachment stores make later retrieval more difficult.

Unguided narrative fields may omit the incident details needed for useful categorisation.

Missing status history prevents the complainant from distinguishing receipt, review, and assignment.

These limitations define the baseline used for requirements analysis. Their severity varies with the procedures and features of the system being compared.

3.3 Proposed System

CyberGuard implements intake, text classification, model-input masking, access control, and complaint tracking behind a shared API.

The React client on port 3000 and Expo client on port 8081 send requests to the Express backend on port 5002. For classification, the backend calls the Flask service on port 5003. MongoDB uses port 27017 in the documented topology. The selected inference pipeline converts masked text into word-level TF-IDF features and returns one of thirteen labels.

PrivacyShield replaces supported identifiers before the inference request. Backend role checks control

retrieval and modification of the complaint, including access to sensitive fields. Escalation rules inspect the resulting category and selected complaint values; Socket.IO then distributes permitted status and message events to connected clients.

The proposed workflow changes three handovers. Intake produces a preliminary category rather than only storing text. Staff retrieve a permission-filtered record. Complainants retrieve status history instead of depending entirely on a separate enquiry. These are implemented capabilities; reductions in workload or response time require a separate operational study.

Table III retains the report's reconciliation of earlier documentation with its stated repository audit. It records the project's supplied evidence; the repository and evaluation artefacts are not reproduced in this report.

Table III: Reconciliation of Document and Repository Claims

Conflict Claim	Previous / Outdated Status	Verified Resolution (Latest Codebase)
Dataset size	Docstring said 500+; tracked CSV had 257 rows	Corpus expanded to 1,950 narratives; generator and CSV agree.
Class balance	Classes ranged from 9 to 38 examples (imbalanced)	Perfect balance: exactly 150 samples per class across all 13 labels.
Taxonomy	README: 11; classifier: 13; schema: 17	Single canonical 13-label taxonomy; schema display mapped from the canonical labels.
Model accuracy	API returned a hard-coded 96.5% fallback	Hard-coded fallback removed; measured results reported separately: 79.48% ± 3.21% 5-fold GroupKFold CV (Macro-F1 = 0.7507); held-out accuracy 88.97%, Macro-F1 = 0.8896.
Trained artefacts	No .pkl tracked; fresh checkout failed to load	Trained classifier and vectorizer artefacts versioned in ml-service/models.

3.4 Functional Requirements

The functional requirements define externally observable operations rather than internal library features.

Require authentication before granting access to protected application functions.

Accept the incident details and narrative submitted by a complainant.

Associate permitted evidence files with the complaint after upload validation.

Return a preliminary category or fallback outcome for the submitted description.

Replace supported identifier patterns with tokens before model inference.

Restrict record visibility and permitted actions according to the authenticated account's role.

Provide complaint status and history to users who are authorised to view the record.

Apply configured escalation conditions to safety-related complaints.

Deliver supported status and message events to connected, authorised clients.

Allow authorised staff to review complaints and perform valid status transitions.

Verify authentication tokens on protected requests and enforce their configured expiry.

Validate required fields, permitted file types, and upload sizes on the backend.

Record complaint creation and subsequent status or priority changes in the case history.

Associate classification and other significant workflow events with the complaint record.

3.5 Non-Functional Requirements

The non-functional requirements state the intended operating qualities. They are design targets unless Chapter 6 reports a corresponding measurement.

Usability: make the guided form understandable to a first-time complainant; confirm this through future user testing.

Responsiveness: keep submission, inference, and retrieval practical under the intended load; establish limits through load testing.

Security: enforce authentication, server-side authorisation, request validation, and restricted file access.

Privacy: mask supported identifiers at the model-input boundary and protect raw stored narratives separately.

Fault visibility: expose service failures and fallback outcomes rather than presenting them as normal model results.

Maintainability: keep client views, API handlers, persistence, and model inference independently modifiable.

Deployment flexibility: permit application and inference components to be updated without rewriting their shared contract.

Compatibility: support the documented browser and mobile environments.

Data minimisation: avoid sending the raw narrative to inference and avoid unnecessary identifier logging.

Availability: continue complaint intake through the keyword fallback when the classification service is unavailable.

3.6 Hardware Requirements

The documented development setup uses a general-purpose computer rather than an accelerator. It must run the client tools, Express API, MongoDB, and Flask service together. A compatible phone is needed for mobile testing, and network connectivity is needed between components when they run on different devices. Table 3.1 records the working hardware recommendation.

Table 3.1: Hardware Requirements

Component	Requirement
Processor	Modern multi-core processor
RAM	8 GB recommended for development
Storage	Sufficient space for source code, dependencies, database, and evidence files
Display	Standard monitor or laptop display
Network	Internet or network connectivity for services requiring communication
Mobile Device	Compatible smartphone for testing the mobile application

Table 3.1 is a development recommendation, not a measured production capacity estimate. Evidence volume, simultaneous users, and hosting arrangements determine the resources needed for deployment.

Model training runs offline. During application use, the Flask service transforms a complaint and predicts its category using loaded artefacts. This workload does not imply a GPU requirement, but response time and concurrent capacity still need measurement on the intended host.

3.7 Software Requirements

We assigned each library a concrete role. React renders the submission and tracking views; React Native with Expo supplies the mobile interface. Express handles authenticated API requests, Flask exposes Python inference, and scikit-learn loads the vectorizer and classifier. Mongoose defines the MongoDB

record structures, Socket.IO carries events, and Tesseract.js extracts candidate text from evidence images. Table 3.2 lists these dependencies.

Table 3.2: Software Requirements

Software / Technology	*Purpose*
React	Complaint forms, tracking views, and staff dashboard components on port 3000
React Native / Expo	Mobile submission and tracking through the Expo client on port 8081
Node.js	Runtime for the Express backend on port 5002
Express.js	Resource routes, JWT checks, request validation, and complaint handlers
Python	Offline training and inference code for the complaint classifier
Flask	HTTP interface to the classifier on port 5003
Scikit-learn	Word-level TF-IDF and Logistic Regression (C=2.0, balanced weights, lbfgs)
MongoDB	Complaint and related record persistence on port 27017
Mongoose	Schemas and model operations for application-managed MongoDB records
Socket.IO	Case-linked status, priority, and message events
HTML, CSS and JavaScript	Browser form markup, styling, and client behaviour
Git	Version tracking for the application and model-service code
Tesseract.js	Candidate text extraction from evidence images before regex review

The requirements establish the tasks each component must perform and the risks that remain outside the current evaluation. Chapter 4 maps these requirements to services, data flow, access checks, and the inference pipeline.

The important boundary is between application coordination and statistical inference. Express owns the complaint workflow; Flask receives the masked narrative needed to suggest a label. Neither framework supplies privacy, authorisation, or reliable classification automatically. Those behaviours depend on the project's handlers, configuration, and tests [11], [12].

CHAPTER 4
SYSTEM DESIGN AND METHODOLOGY

4.1 System Architecture

CyberGuard separates presentation, application coordination, inference, and persistence. React and React Native clients call the Express API; the API validates requests, enforces permissions, and manages complaint records. A server-to-server HTTP call reaches the Flask classifier. Mongoose handles application-level document modelling, while Socket.IO provides the event channel [11], [12], [17], [19].

The Flask service loads the trained TF-IDF vectorizer and Logistic Regression artefacts at startup. It receives the masked narrative, computes features in the saved vocabulary, and returns a proposed category with confidence information. The selected configuration is word-level; character and combined feature spaces belong to the candidate evaluation, not the deployed configuration.

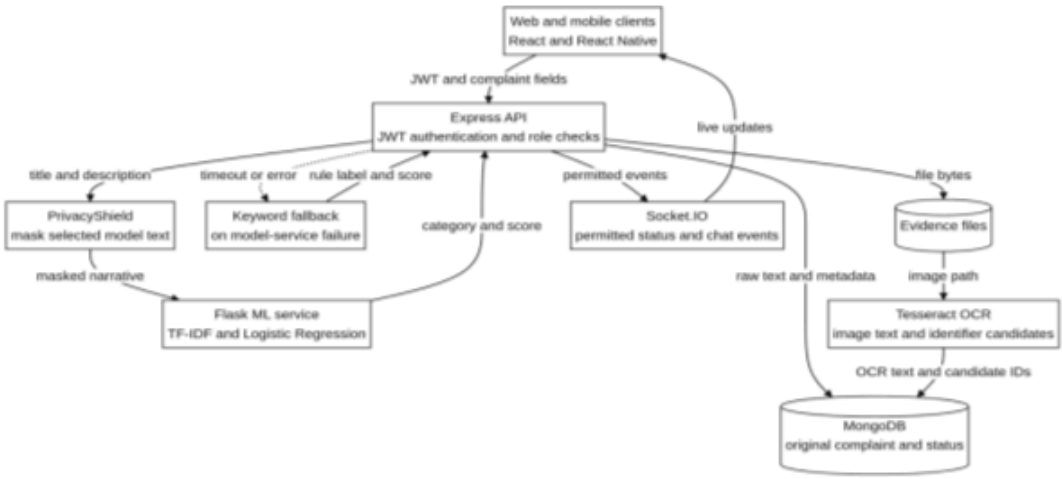


Figure 4.1 System Architecture of CyberGuard

A separate inference service allows the model artefacts to change without restructuring the client API. If the Flask call fails, the backend invokes its keyword fallback and records the alternative path. This preserves the intake workflow, not equivalent prediction quality. The local topology uses port 5002 for Express and port 5003 for Flask.

4.2 System Workflow / Block Diagram

The client submits incident details and attachments to the backend. Request validation checks required fields and upload restrictions. PrivacyShield then constructs the masked text that will be used for classification; the original narrative remains sensitive stored data.

The classifier returns a proposed category unless inference fails or the confidence gate rejects the input. The fallback and review path handle those cases. Escalation rules inspect the category and selected fields, and the complaint record retains category, priority, and timeline information. Authorised users subsequently retrieve the record and exchange case-linked updates.

Safety-related escalation occurs during intake so that the initial record can already carry a priority flag. The flag requests attention; it does not certify urgency or replace an officer’s assessment.



Figure 4.2 Activity Diagram for Cybercrime Complaint Processing

This ordering defines two important checks. The inference request must use masked rather than raw narrative text. Workflow events must also be associated with the stored complaint so that staff can inspect how its category, status, or priority changed. A timeline is an application audit aid, not a cryptographic guarantee of evidence integrity.

4.3 Module Description

The implementation is divided into the modules described below.

4.3.1 User Interface Module

The client module presents the submission form, evidence upload, complaint list, tracking history, and staff detail views. Web and mobile clients consume the same backend contracts. They display permitted actions, but access decisions remain on the server.

The screens reflect the user's task. A complainant enters an incident and tracks their own reports. An investigator reviews assigned cases. An administrator uses lists and aggregate views. Safety-related queues provide a focused review view. Separating these views improves navigation; it does not replace backend field and record checks.

4.3.2 Authentication and Access Control Module

The authentication module issues a signed token after successful login. Protected handlers verify that token and apply the caller's role before returning or modifying a record. The client's visible controls are therefore not the authority for access decisions [11].

4.3.3 Complaint Management Module

Complaint handlers create and retrieve records and apply permitted status changes. Status values are constrained by the application rather than entered as arbitrary text. This supports consistent filtering and

counts while keeping a history of the transitions performed by authorised users.

4.3.4 Evidence Management Module

Evidence handling links uploaded files to the parent complaint and applies upload type and size restrictions. Access follows the parent record's permissions. These checks address file handling; malware inspection, authenticity, and evidentiary admissibility require additional mechanisms [20].

4.3.5 PrivacyShield Module

PrivacyShield tests narrative text against regular expressions for supported identifiers, including phone numbers and e-mail addresses. It substitutes numbered tokens such as PHONE_1 and EMAIL_1. A pattern does not recognise every way an identifier can be expressed, so the output remains potentially sensitive.

The backend applies this substitution before constructing the model request. A new supported format requires a corresponding detection rule and tests. Raw narrative retention and evidence files remain outside the protection provided by this inference-text transformation.

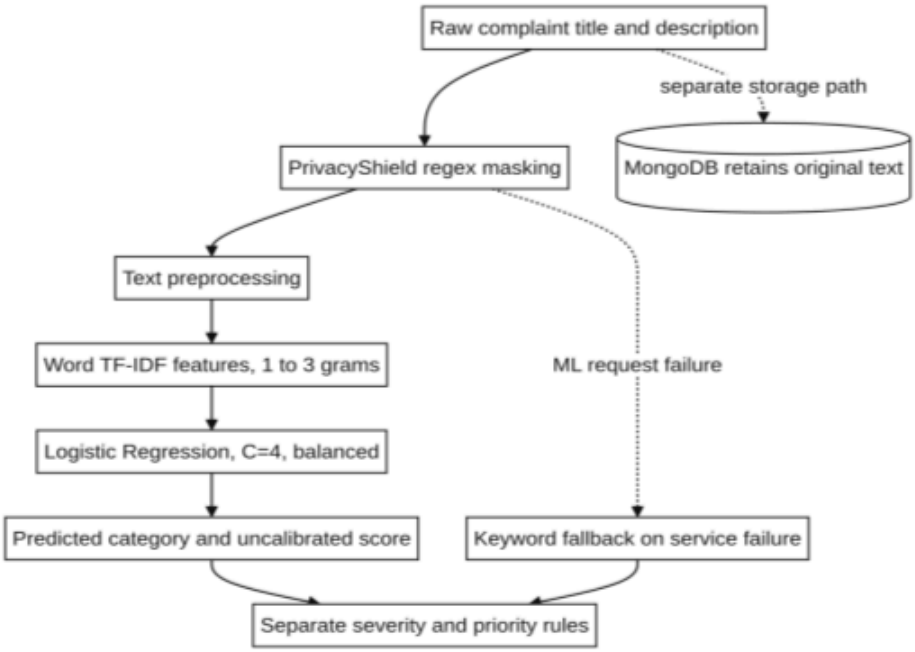


Figure 4.3 PrivacyShield Model-Input PII Masking Pipeline

4.3.6 Machine Learning Classification Module

The classification module maps the masked narrative into word-level TF-IDF features and applies Logistic Regression. It returns a category and probability estimates; the tau = 0.20 gate determines whether the model output is accepted or referred to the fallback and review path.

Inference uses loaded artefacts rather than training on incoming requests. Complaint records remain the responsibility of the main backend and database. Keeping the service contract narrow avoids unnecessary data exchange, but retention and logging policies still require explicit controls.

4.3.7 Priority Escalation Module

The escalation module evaluates configured conditions against the category and selected complaint fields.

A matching condition raises priority or severity for review. Personnel remain responsible for judging the incident and deciding the appropriate response.

4.3.8 Communication and Notification Module

The notification module emits Socket.IO events for messages, status transitions, and priority changes. Recipients must be associated with the permitted complaint or role channel before receiving those updates [19].

Connected clients receive an event when a relevant change occurs. The database history serves a different purpose: it provides the record on a later retrieval. A persistent timeline is needed because a live event channel alone does not establish durable delivery to an offline user.

4.4 Data Flow / Process Flow

The logical processing path has the following stages:

Receive the complainant's incident fields and attachments.

Validate required values and the configured upload restrictions.

Replace supported identifiers in the narrative prepared for inference.

Request a category and confidence result, or use the fallback and review path.

Evaluate escalation conditions against the category and selected complaint fields.

Persist the complaint metadata and associated initial timeline information in MongoDB.

Allow authorised staff to retrieve the record and perform permitted workflow actions.

Record subsequent changes and notify connected, permitted users through Socket.IO.

The dependencies are explicit: masking precedes the inference request, and category-based escalation requires a category outcome. The diagram expresses the logical flow, not a guarantee that all database writes occur atomically. Transaction handling and recovery after a partial write need separate implementation verification.

4.5 Database Design

MongoDB stores the application records, and Mongoose schemas define the structures used by the backend [14], [17]. The documented collections are users, complaints, evidence, timelines, and messages. These names are retained throughout the design; the report does not introduce a new schema or assume an unreported index.

The entities store the following information:

users: account information, credential hashes, and role assignments.

complaints: raw and masked narratives, category, status, priority, and incident metadata.

evidence: file references associated with a complaint.

timelines: ordered entries describing recorded complaint events.

messages: communication linked to the complaint.

Identifiers link complaints to related evidence, timeline, and message records. Mongoose validates the application-managed operations that use its models; it is not a control over every possible direct database write [17]. Backend permission checks govern retrieval. The presence of raw narratives also makes

storage protection a separate requirement from model-input masking.

4.6 Algorithm / Model / Technique Used

4.6.1 TF-IDF Feature Extraction

The vectorizer converts each masked narrative into a sparse feature row. During fitting, it learns a vocabulary and inverse-document-frequency weights from training text. During inference, it applies that saved mapping without adding terms from the new complaint [4].

The deployed settings are word-level TF-IDF with 1–2 n-grams, sublinear_tf=True, strip_accents='unicode', and max_features=10,000. Character n-grams using char_wb and combined word-plus-character representations were evaluated as candidates. Those alternatives must not be described as the selected word-level pipeline.

Unigrams represent individual terms such as UPI or OTP; bigrams preserve adjacent pairs such as customer care. With a 1–2 n-gram range, the deployed vectorizer does not extract word trigrams. The 10,000-feature limit bounds the learned vocabulary, while sublinear term-frequency scaling reduces the effect of repeated words [4].

4.6.2 Logistic Regression Classification

The Logistic Regression estimator learns class-specific weights over the sparse TF-IDF features. At inference it scores the thirteen categories and produces probability estimates through predict_proba [5]. The highest estimate supplies a candidate label. That label remains a routing suggestion rather than a finding about the complaint.

We configured Logistic Regression with C=2.0, class_weight='balanced', and solver='lbfgs'. The model supplies probability estimates for the tau = 0.20 out-of-distribution gate. If the maximum estimate is below the threshold, the input is referred to keyword fallback and human desk review. The probabilities have not been independently calibrated; the threshold is an engineering filter, not a calibrated risk boundary [18]. Training occurs offline, and the service loads the resulting artefacts at startup.

4.6.3 PrivacyShield Pattern-Based Masking

PrivacyShield applies supported regular expressions in a defined order, replaces matched identifiers with type-specific numbered tokens, and returns the transformed text. Unrecognised formats remain in the result. This operation masks model-input text; it neither sanitises every evidence file nor removes the raw narrative retained in the database.

4.6.4 Rule-Based Fallback and Priority Escalation

The backend fallback scores category-specific keywords and short phrases in the masked description and chooses the best-supported category. It is used when the inference service is unavailable or the configured gate rejects a model result. The fallback keeps intake moving, but its contextual understanding and error profile differ from those of the statistical model.

Escalation is a separate rule evaluation over the category and selected complaint values. A match changes priority or severity and records the event for review. The system flags the record; authorised personnel determine its operational handling.

4.6.5 Data Preprocessing and Model Training Pipeline

Preprocessing prepares masked text for the saved vectorizer through the configured normalisation and tokenisation steps. The documented settings include Unicode accent normalisation and sublinear term frequency. No additional stemming or lemmatisation step is assumed merely because it is common in other NLP pipelines. Training and serving must use compatible preprocessing.

The corpus contains 1,950 balanced synthetic complaints across 13 classes (150/class), spanning 130 scenario families, with ten families per class. The development partition contains 1,560 samples in 104 scenario groups (120 per class); the group-disjoint held-out test contains 390 samples in 26 groups (30 per class). These correspond to an 80 percent / 20 percent split with no shared scenario groups. Model selection uses 5-fold GroupKFold on the development partition [6]. The reported protocol fits the vectorizer on development rather than held-out data. For valid fold-level estimates, vocabulary and IDF fitting must also be restricted to each training fold; scikit-learn Pipeline supports that separation [8].

The trained vectorizer and classifier must be retained as a compatible inference bundle. The Flask service loads the saved artefacts at startup rather than refitting them on requests. Retraining is an offline operation followed by evaluation and deployment of the replacement bundle; changing the model without its matching vocabulary would invalidate inference.

4.7 Methodology

Implementation proceeded around the main complaint path: client submission, authenticated API handling, storage, model inference, privacy processing, and status updates. Each service has a defined input and output, which permits module checks as well as end-to-end testing.

The backend calls Flask over HTTP for classification and applies PrivacyShield before that call. Role checks remain in the request path for complaint access. This separates model processing from the application’s authority to read or modify a record.

Integration testing must examine interactions as well as individual outputs. In particular, identifier replacement changes the feature text seen by the classifier, so a model trained on raw narratives may not behave identically when served masked narratives. The final test scope therefore includes the masked inference path, fallback behaviour, category mapping, and escalation. The report does not claim an ablation result for masking unless such a comparison is measured.

Tests are organised around observable module behaviour: a request is accepted or rejected, a category is returned, an identifier is replaced, a record is filtered, or an update becomes retrievable. Cross-component tests then check that these outcomes remain consistent along the full complaint path.

Figure 4.4 retains the project methodology diagram. It places client intake, backend coordination, privacy processing, classification, storage, communication, and staff review within the same development workflow.

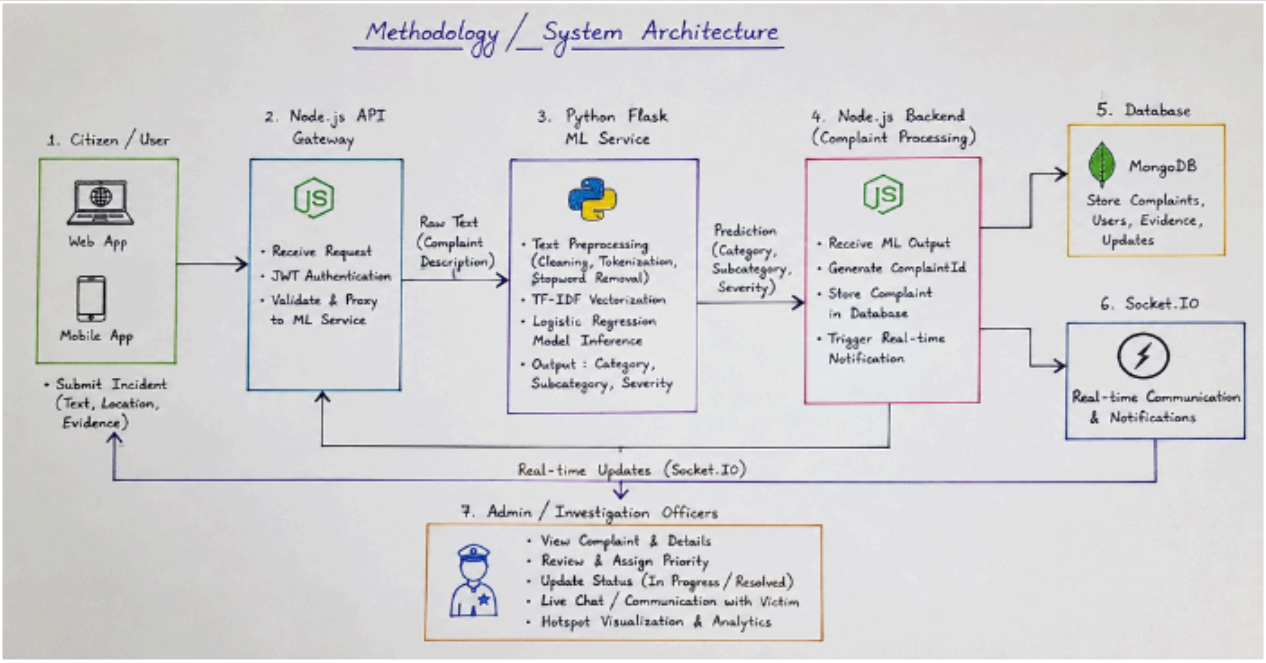


Figure 4.4 Methodology of CyberGuard

This chapter defines the service boundaries, processing order, record entities, and inference configuration. Chapter 5 describes their implementation and the interfaces used to exercise them.

CHAPTER 5
IMPLEMENTATION

5.1 Development Environment

The development setup contains four application components: React web, React Native/Expo mobile, Node.js/Express API, and Python/Flask inference. MongoDB supplies persistence. The documented local ports are 3000, 8081, 5002, 5003, and 27017, respectively for web, mobile, API, inference, and database. This topology describes the prototype setup, not a production deployment specification.

The repository separates web, mobile, backend, and ML-service code. Git tracks the components together while their service interfaces remain distinct. This organisation lets a client view change without moving classification code into the client or database logic into the model service.

Table II records the local service topology; Table 5.1 lists the technologies used in the implementation.

Table II: System Technology Stack and Service Topology

Architectural Layer	**Technologies and Specifications**	**Operational Role**
Web Client	React.js, Tailwind CSS, Lucide Icons (Port 3000)	Complaint forms, tracking, and officer/admin views in the local web client.
Mobile Client	React Native, Expo SDK 54, Expo Router (Port 8081)	Mobile intake and the documented device-scan, Threat Radar, and biometric-login interfaces; validation depth varies by feature.
Backend API	Node.js, Express, REST, JWT, Socket.IO (Port 5002)	Request validation, JWT/RBAC checks, timeline handling, and evidence routing.
Database	MongoDB / Mongoose ODM (Port 27017)	Complaint records, user accounts, timelines, audit information, and raw/masked narrative fields.
ML Microservice	Python, Flask, scikit-learn, TF-IDF (Port 5003)	Server-to-server category inference; backend connector supplies keyword fallback.
Evidence / OCR	Local File Storage, Tesseract.js	Local evidence references, candidate OCR text, and regex-based identifier extraction.

Table 5.1: Technologies Used in CyberGuard

Technology	Application in CyberGuard
React	Submission, tracking, and role-specific web views
React Native and Expo	Mobile intake and complaint-tracking views
Node.js	Runtime for the API and complaint workflow
Express.js	Resource routes and request middleware on port 5002
Python and Flask	Offline training and inference service on port 5003
Scikit-learn	Saved word-level TF-IDF and Logistic Regression estimators
MongoDB	Complaint and related record persistence
Mongoose	Application schemas and document operations
Socket.IO	Status, priority, and case-message events
Git	Version tracking across client and server components

5.2 Implementation Details

The implementation keeps the service contracts narrow. Clients submit incident fields and render responses. Express validates requests, applies permissions, prepares inference text, coordinates workflow changes, and writes through Mongoose. Flask transforms the masked narrative and returns a classification outcome. MongoDB stores the application records.

Table I: Implemented Components and Feature Boundaries

System Feature	Repository Implementation	Operational Role and Security Boundary
Web and Mobile Intake	React web + Expo React Native	Guided incident reporting and complaint-linked evidence upload.
Authentication and RBAC	JWT bearer tokens + role guards	Role guards separate citizen, officer, and admin actions on protected requests.
PrivacyShield Anonymizer	Regex tokenization in Express and Python	Supported e-mail, phone, and UPI patterns are tokenised before model input; raw storage is not anonymised.
Text Classifier	Word-level TF-IDF + Logistic Regression; word/char alternatives evaluated	Produces a proposed label among 13 categories and a probability estimate.
Keyword Fallback	Rule-based dictionary in the ML connector	Keeps category routing available on a Flask timeout, with review of fallback outcomes.
Evidence OCR and Regex	Tesseract.js engine + Indian regex rules	Produces candidate phone, UPI, and account identifiers for review, not verified suspect facts.
Real-Time Socket Events	Socket.IO rooms and event channels	Delivers complaint-related notifications to connected, permitted recipients.

The client collects the incident narrative and attachments, then submits the payload to the backend. A successful response supplies confirmation and a tracking reference. The client does not decide whether a category is accepted, whether a case is escalated, or whether another account may read the record.

Express routes are grouped around authentication, complaints, evidence, timelines, and messages [11]. Protected handlers verify the JWT before performing a permitted operation. Complaint creation coordinates validation, masked-text preparation, the classifier or fallback call, escalation, and persistence of the initial history. These are logical processing steps; the report does not claim a transaction guarantee for all related writes.

Mongoose models define the structures used for backend database operations [17]. The stored complaint associates its narrative with category and workflow metadata. Flask is called over HTTP, and the returned result is attached to that record rather than stored as an independent case in the inference service.

PrivacyShield, role checks, and escalation belong to the backend request path. Socket.IO provides the event channel after authentication. Transporting an update and authorising its recipient are separate requirements; a connected client must not receive a complaint merely because it has a socket connection.

The resulting implementation connects intake, preliminary categorisation, controlled access, and progress tracking without treating any one module as the complete complaint-handling process.

5.3 Module-wise Implementation

5.3.1 User Interface Implementation

The React and Expo clients expose submission, tracking, complaint lists, and role-specific detail views. They use shared JSON contracts with the backend. React components render the relevant form or record state [9], while server-side handlers remain responsible for validating the submitted data and deciding

what the caller may access.

5.3.2 Authentication and Authorization

A successful login returns a signed authentication token. Passwords are stored as hashes rather than plain text. Subsequent protected requests carry the token, and the backend verifies it before applying role restrictions. No unreported hashing algorithm, token lifetime, or session policy is assumed here.

Authorisation is enforced in the backend, not inferred from which buttons the client displays. Complaint retrieval and modification must check the authenticated caller against the permitted role and record. A hidden interface control is only a presentation choice; it does not prevent a direct API request.

5.3.3 Complaint Submission and Management

The complaint API creates a record with its category, status, and priority, and exposes permitted updates through the workflow handlers. Staff list views query the same complaint data with the appropriate access and status filters. A category supplied by the classifier remains open to review rather than becoming an immutable assertion.

Status transitions use defined application values rather than unrestricted prose. The workflow includes submission, review, and assignment stages. Keeping those values controlled supports consistent list filters and stage counts, while the timeline retains the changes made to the record.

5.3.4 Evidence Management

The evidence handler associates accepted uploads with the complaint and applies the configured type and size restrictions. Retrieval is subject to the parent complaint's access rules. These controls do not establish that a file is harmless, genuine, or legally admissible; stronger upload inspection remains a separate requirement [20].

Files are stored separately from the complaint document and reached through recorded references. Tesseract.js supports OCR of evidence images, followed by regex extraction of candidate identifiers [10]. OCR does not redact the original file or validate its contents. Comprehensive forensic analysis and cryptographic custody tracking are outside the demonstrated implementation.

5.3.5 PrivacyShield Implementation

PrivacyShield applies supported regular expressions and substitutes numbered identifier tokens, including PHONE_1 and EMAIL_1. The tokenised narrative is used in the inference request. Detection depends on the written patterns, so an unsupported number format or an identifier expressed in words can remain undetected.

The demonstrated privacy boundary is the model-input path, not complete anonymisation. Raw narratives remain in MongoDB, and uploaded files retain their original contents. Backend permissions and careful logging are therefore still required. Future validation must measure missed identifiers and unintended substitutions, not simply confirm that a familiar phone-number pattern was replaced.

5.3.6 Machine Learning Classification

At startup, Flask loads the saved vectorizer and Logistic Regression artefacts. A request transforms the masked narrative using the fitted vocabulary, obtains category probabilities, and applies the configured

gate. The backend uses its keyword fallback when inference fails or the result is referred for alternative handling, and the outcome remains subject to staff review.

Canonical classifier labels are mapped to the interface's category views. Financial complaints appear in the relevant desk view, while configured safety categories enter the Women and Child Safety review path. This is an application routing rule. It does not establish jurisdiction or confirm the legal character of the incident.

5.3.7 Priority Escalation

The escalation handler evaluates its conditions after the category outcome is available. A match raises the configured priority or severity and makes the case visible in the corresponding review queue. The rule flags the complaint for attention; the responsible personnel decide the response.

5.3.8 Real-Time Communication

Socket.IO emits supported events for new case messages, status changes, and priority escalation. The client establishes its authenticated subscription during a session and removes it on logout. Case and role permissions must govern the rooms or channels used for delivery [19].

A connected recipient can see an update without manually refreshing the view. The recorded state remains in the database for later retrieval. This event mechanism is not evidence that every offline client received a notification, nor does it replace a durable message and history store.

5.4 Algorithms / Models Implementation

5.4.1 Complaint Classification Algorithm

The classification path executes the following logical steps:

1. Receive the narrative selected for inference.
2. Apply supported identifier masking and the configured text preprocessing.
3. Transform the text using the fitted word-level TF-IDF vocabulary.
4. Compute Logistic Regression probability estimates for the thirteen categories.
5. Accept the highest-scoring label only if it satisfies tau = 0.20; otherwise refer the input to fallback and review.
6. Return the category outcome and associated confidence or fallback information to the backend.
7. Associate the classification outcome with the complaint record.

The output is a statistical label inferred from the training examples. It supports sorting and review, not legal determination. A high model score does not verify the narrative or guarantee that the report is in distribution.

Masking precedes feature extraction so that the model does not rely on supported raw identifiers in the inference request. Training and serving must use compatible transformations. The report has not supplied an ablation that quantifies the change in accuracy caused by masking, so no such improvement is claimed.

5.4.2 PrivacyShield Masking Algorithm

The masking procedure follows these steps:

1. Read the narrative selected for the model request.

2. Test each supported identifier pattern in the configured order.
3. Locate the matching spans for that pattern.
4. Replace recognised spans with the corresponding numbered tokens.
5. Return the transformed narrative to the calling handler.
6. Use the transformed version, rather than the raw narrative, in the inference request.

The transformation does not affect unsupported identifier formats or the contents of uploaded files. Its output must therefore remain under the application's privacy controls.

A stable masking configuration should produce repeatable output for the same input. Idempotence is a separate property: applying the transformation again must not alter its existing tokens. Both behaviours should be covered by regression tests rather than inferred from the use of regular expressions alone.

5.4.3 Priority Escalation Algorithm

Escalation rules inspect the category and selected complaint fields. A matching condition changes priority or severity; an unmatched case retains its ordinary route. The event is associated with the complaint so staff can inspect the reason for its position in the queue.

Authorised personnel review the flagged record before deciding how an urgent complaint should be handled.

5.4.4 Keyword-Based Fallback Classification

The backend keyword matcher accepts the same masked narrative used for classification and returns a compatible category outcome. It scores configured words and short phrases for each category. Since it resides in the backend connector, it can operate when the Flask service is unavailable.

The fallback is an availability mechanism, not a model-equivalent replacement. It may miss paraphrases, negation, or overlapping incident descriptions. Staff can review the categories it produces, and subsequent requests return to the statistical inference path when that service is available and its result passes the gate.

5.5 User Interface / Hardware Implementation

No dedicated hardware is part of the prototype. Users access the web interface through a browser or the Expo-based mobile client on a compatible device. Submission forms, complaint lists, tracking views, and administrator screens expose the implemented workflow.

5.6 Application User Interface and Implementation Screens

Figures 5.1–5.6 retain the supplied localhost browser screenshots. They document the prototype's citizen, officer, and administrator views. A rendered control demonstrates its presence in the interface; it does not, by itself, verify a bank action, official filing, statutory compliance, or production deployment.

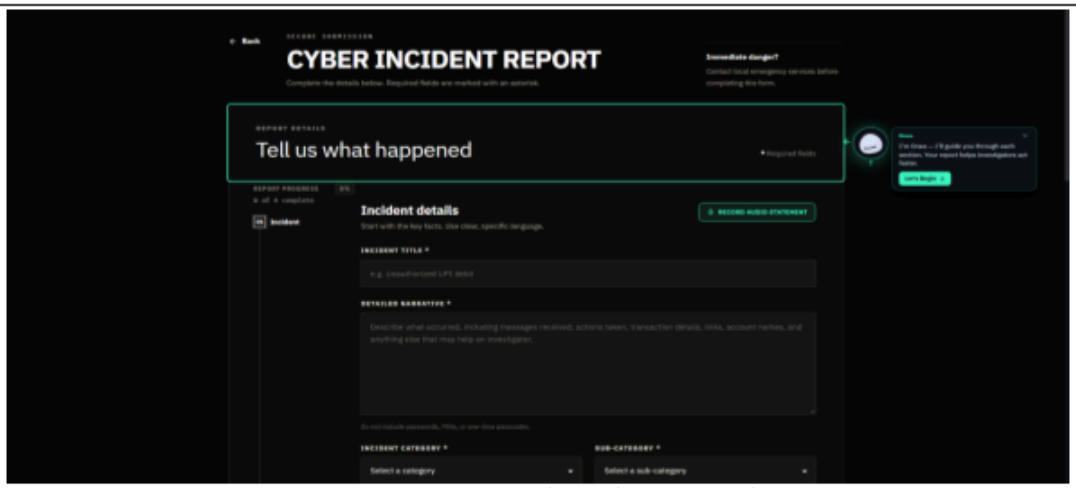


Figure 5.1 Citizen Complaint Submission Portal

The submission screen collects the incident title, narrative, category, and sub-category. It also displays an audio-statement recorder and a progress indicator across four sections. The narrative is the input used for masking and classification. The masking banner describes that intended processing path; it should not be interpreted as proof that the database or attached files contain no personal information.

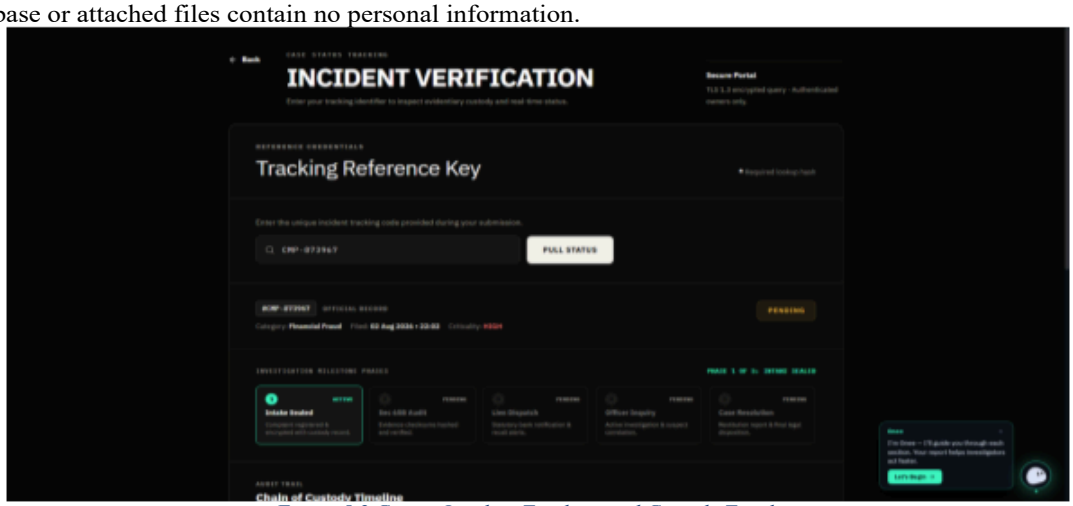


Figure 5.2 Citizen Incident Tracking and Custody Timeline

The tracking screenshot displays reference #CMP-873967, category Financial Fraud, filing date 02 Aug 2026, criticality HIGH, and status PENDING. Its milestone labels run from Intake Sealed through Sec 65B Audit, Lien Dispatch, Officer Inquiry, and Case Resolution. These five labels and the timeline are reproduced as interface content, not as a verified statutory sequence or proof of a legally complete chain of custody.

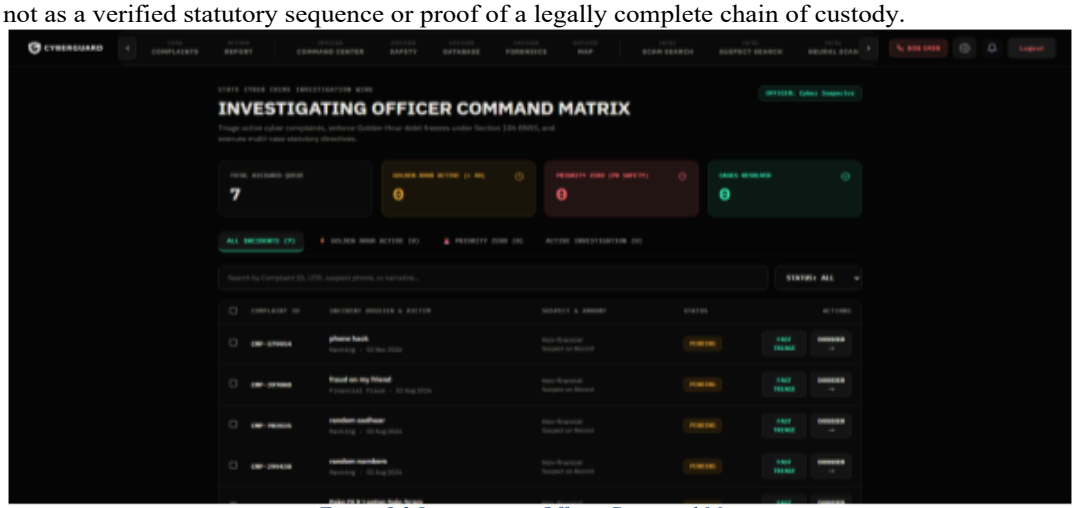


Figure 5.3 Investigating Officer Command Matrix

The officer view shows an assigned queue of 7, a Golden-Hour counter for the four-hour window, a Priority Zero safety counter, and resolved totals. The table includes suspect and amount columns, status indicators, and fast-triage and dossier actions. The screen's Section 106 BNSS reference and debit-freeze controls are prototype interface elements; the screenshot does not establish that a bank freeze was executed or legally authorised.

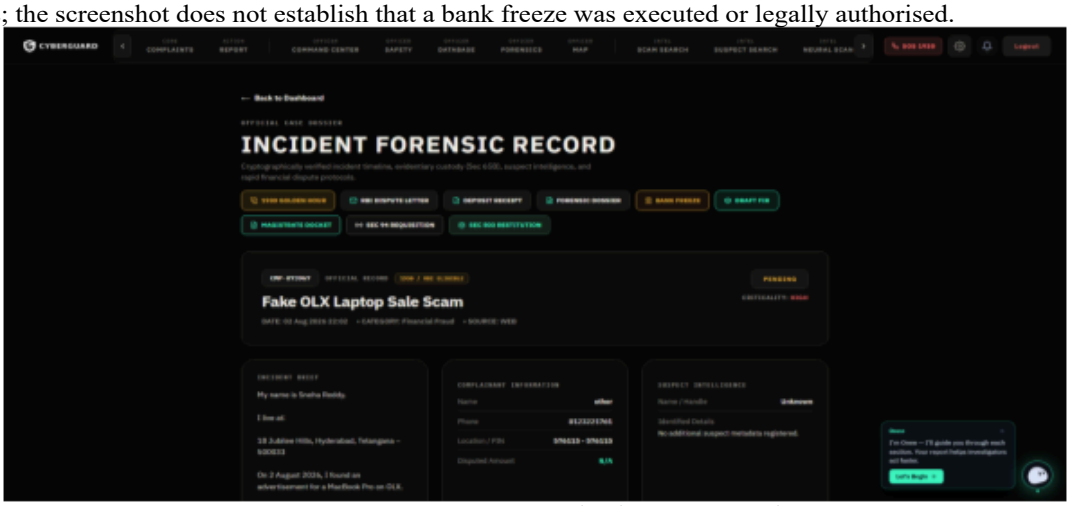


Figure 5.4 Case Forensic Record and Dispute Protocols

For the Fake OLX Laptop Sale Scam record (CMP-873967), the interface groups controls labelled 1930 Golden-Hour freeze, RBI dispute letter, deposit receipt, forensic dossier, bank freeze, draft FIR, magistrate docket, Section 94 requisition, and Section 903 restitution. Incident, complainant, and suspect panels appear alongside them. The section labels are retained as displayed, not endorsed as correct legal citations. Their legal basis and any external action require separate professional verification.

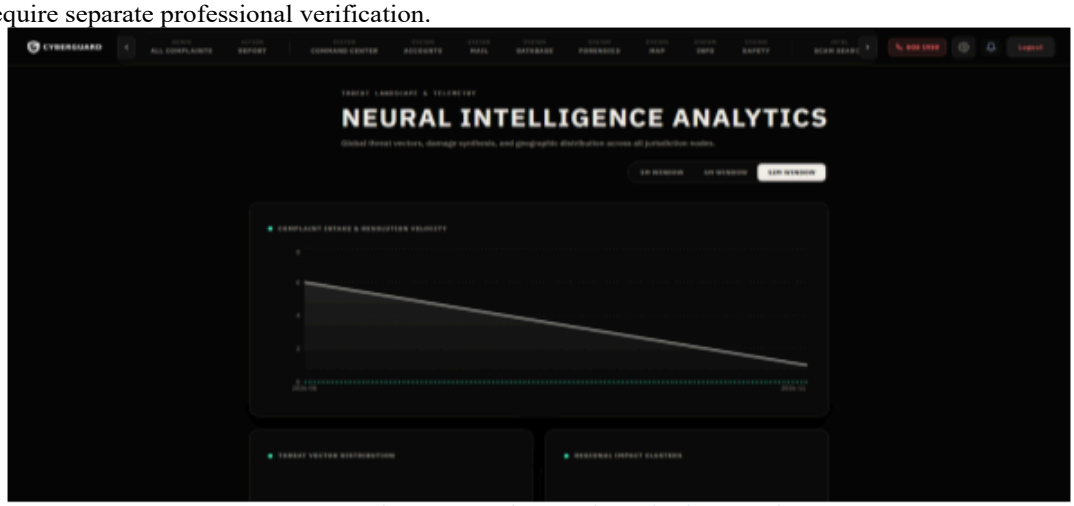


Figure 5.5 Administrative Threat Radar and Velocity Analytics

The administrator screen plots complaint intake and resolution velocity for selectable 3, 6, and 12-month windows. It also displays threat-vector distribution and regional impact clusters. These views summarise the records available to the application; they do not establish a wider regional crime trend or measure the completeness of reporting.

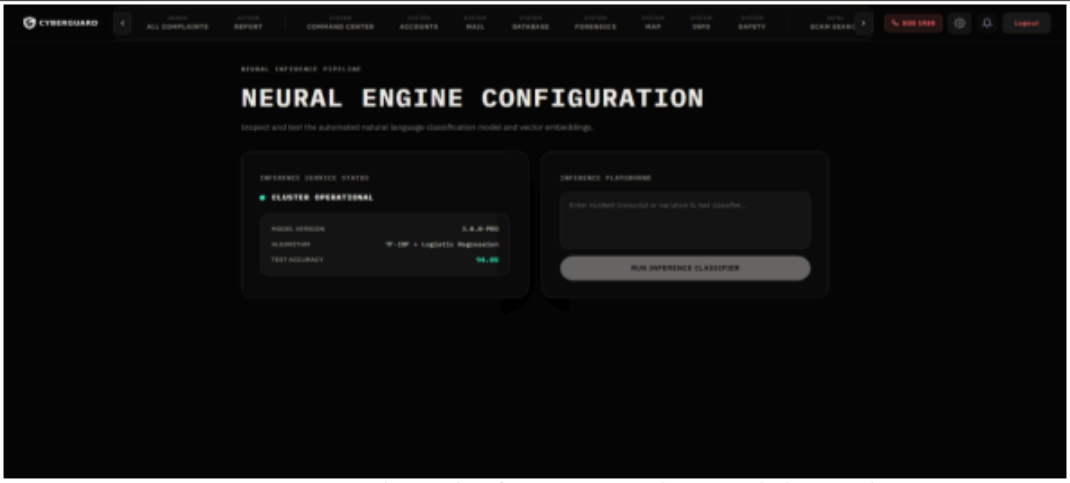


Figure 5.6 Statistical NLP Classification Engine Telemetry and Playground

The engine configuration screen displays inference-service status, model version, algorithm information, and an interactive classification playground. Although the interface uses a “neural engine” label, the selected classifier reported here is word-level TF-IDF with Logistic Regression, not a neural network. The status display is telemetry for the prototype service.

The implementation connects clients, API handlers, persistence, masked-text inference, role restrictions, and event delivery. The screenshots illustrate those interfaces, while the evaluation in Chapter 6 separately addresses observed workflow behaviour and model performance. Neither source alone establishes production security or legal readiness.

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Experimental Setup / Test Cases

Testing has two distinct scopes. Functional checks exercise submission, evidence attachment, classification, masking, access restrictions, tracking, and communication through the clients. Cross-component checks follow a request from the form to its stored result, including category mapping and escalation. Model evaluation uses labelled data and a group-aware split, reported separately because an accepted API request does not imply a correct prediction.

Table 6.1 records the principal functional test cases and expected outcomes. It is a test-scope summary, not a complete execution log: the table contains no observed-result, timestamp, or pass/fail column.

Table 6.1: Functional Test Cases

Test ID	Feature	Expected Result
TC-01	User Login	Valid users can access permitted functions
TC-02	Complaint Submission	Complaint details are accepted and stored
TC-03	Evidence Upload	Supporting files can be associated with complaints
TC-04	Complaint Classification	The system predicts a complaint category
TC-05	PrivacyShield	Supported personal information is masked in selected text
TC-06	Role-Based Access	Access is restricted according to user permissions
TC-07	Priority Escalation	Specified safety-related cases receive elevated priority
TC-08	Complaint Tracking	Authorized users can view relevant complaint status
TC-09	Real-Time Communication	Supported messages or updates are delivered to connected users

6.2 Results

The supplied report describes the following functional observations. Figures 5.1–5.6 show the corresponding localhost interfaces. Those captures support interface inspection, while reproducing each end-to-end result would also require the application, test inputs, and execution records.

The reported submission check accepted incident fields and evidence and returned a tracking reference (Figure 5.1). The tracking view resolved a reference into its complaint status, criticality, displayed milestones, and history (Figure 5.2). The milestone names are application labels rather than a validated statement of statutory procedure.

The officer view displayed its assigned queue and the Golden-Hour and Priority Zero counters, with triage and dossier controls linked to case records (Figure 5.3). The financial-fraud detail view exposed the protocol controls shown in Figure 5.4. Their presence does not verify execution of the named banking, police, or court actions.

The administrator view rendered intake and resolution plots, category distribution, and regional clusters for selectable windows (Figure 5.5). The classification page displayed service status, loaded model information, and a narrative playground (Figure 5.6). The classifier evaluated below is word-level TF-IDF with Logistic Regression.

The reported integration checks found masked text at the classifier boundary, category values on

complaint records, safety cases in the priority view, and role-dependent record visibility. These observations address the exercised paths. They are not an exhaustive privacy audit, penetration test, or evaluation of all identifier formats.

6.3 Performance Evaluation

Functional evaluation asks whether the workflow performs its specified operations. Statistical evaluation asks whether predicted labels agree with known labels on separated data. The latter needs an explicit corpus, partition, scoring method, and configuration; a working screen cannot substitute for that evidence.

The model corpus contains 1,950 balanced synthetic complaints across 13 classes (150/class), spanning 130 scenario families, with ten families per class. The labels are phishing, financial fraud, identity theft, cyberbullying, hacking, ransomware, data breach, online fraud, child exploitation, women and child safety, network attacks, crypto fraud, and other. Figure 6.1 shows the equal support. Synthetic text permits controlled development without redistributing real complainant narratives, but it does not establish performance on genuine submissions.

Fig. 3 Distribution of 1,950 synthetic narratives across 13 classifier labels

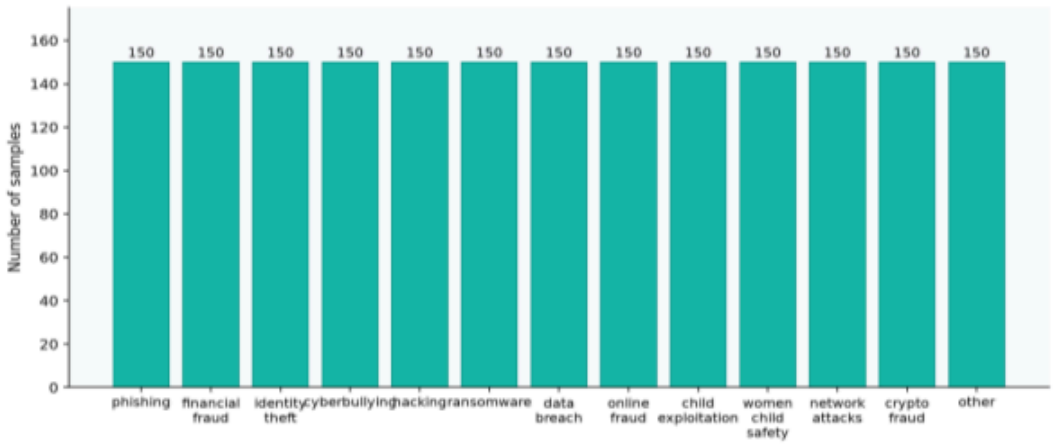


Figure 6.1 Class Distribution of the 1,950-Narrative Corpus across 13 Classes

The evaluation uses the following classification measures [7]:

Accuracy: correctly predicted labels divided by the number of evaluated complaints.

Precision: correct predictions for a class divided by all predictions assigned to that class.

Recall: correctly identified members of a class divided by its true support.

F1-score: the harmonic mean of class precision and recall; macro-F1 averages class F1-scores equally.

Overall accuracy does not express the consequence of an error. Missing a safety-related complaint can be more serious than confusing two financial categories. Class recall and the confusion matrix therefore matter alongside the aggregate score. Similar vocabulary can place related categories near the same decision boundary; the matrix shows which label pairs require closer review.

Precision measures the reliability of assignments to a queue, while recall measures how many relevant complaints reach it. Macro-F1 gives each category equal weight rather than allowing a large class to dominate the summary [7]. Even in this balanced corpus, class-level values are needed because equal

support does not imply equal difficulty.

The development partition contains 1,560 samples in 104 scenario groups, with 120 samples per class. The group-disjoint held-out partition contains 390 samples in 26 groups, with 30 per class. No scenario group is shared between them. Development uses 5-fold GroupKFold [6]. Table IV retains the candidate-comparison run; the selected configuration's separately reported cross-validation result is distinguished below.

Table IV: Candidate Classifiers and Evaluation Metrics (5-fold GroupKFold cross-validation)

Candidate Model	**CV Accuracy (5-fold GroupKFold)**	**Macro F1 (CV)**	**Avg. Predict (ms)**	**Role**
LinearSVC (C=2.0)	81.79% ± 2.29%	0.7869	1.1	1.1 ms; no native probability output
Logistic Regression (C=2.0, Selected)	79.88% ± 3.15%	0.7600	1.6	Native probability estimates for gating; empirical calibration not established
MultinomialNB (alpha=0.1)	76.92% ± 2.18%	0.7317	0.4	Baseline reference

Table IV records one candidate-comparison run: LinearSVC (C=2.0) obtained 81.79% ± 2.29% accuracy, Macro-F1 = 0.7869, and 1.1 ms average prediction time; MultinomialNB (alpha=0.1) obtained 76.92% ± 2.18%, 0.7317, and 0.4 ms; Logistic Regression (C=2.0) obtained 79.88% ± 3.15%, 0.7600, and 1.6 ms. The selected configuration is reported separately at 79.48% ± 3.21% 5-fold GroupKFold CV accuracy and Macro-F1 = 0.7507 (fold dispersion ± 0.0383), with 79.49% out-of-fold accuracy. These are distinct reported runs, not interchangeable scores. The source does not fully establish why the two Logistic Regression runs differ. Logistic Regression was retained for its probability output and tau = 0.20 gate, not for the highest candidate accuracy. Native probabilities do not establish calibration [18]. Figure 6.3 retains the candidate comparison.

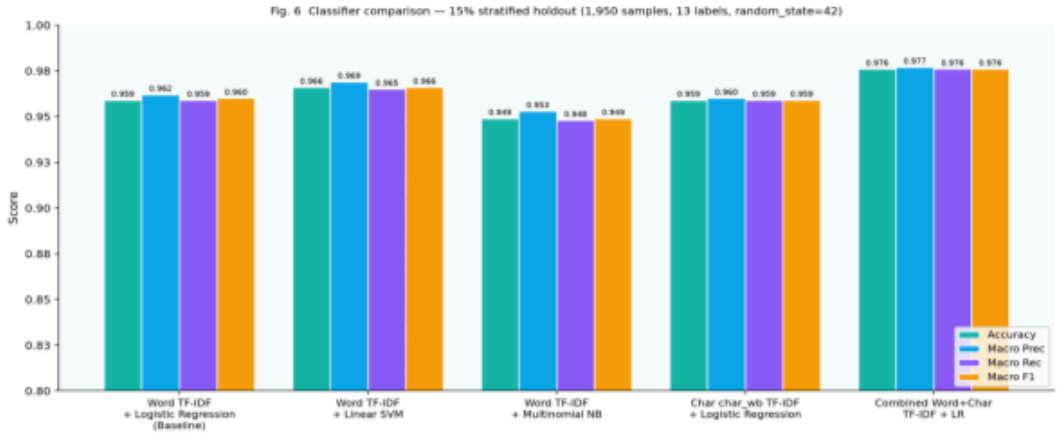


Figure 6.3 Candidate Classifier Comparison (5-fold GroupKFold cross-validation)

On the held-out test (N=390), the selected model obtained 88.97% accuracy (347 / 390 correct), macro precision 0.9016, macro recall 0.8897, and Macro-F1 = 0.8896. The clean subset contained 377 samples and scored 88.86%; the noisy subset contained 13 and scored 92.31%. Class F1-scores were 1.000 for phishing and ransomware, 0.9836 for network attacks, 0.9831 for crypto fraud, 0.9355 for identity theft, 0.8966 for child exploitation, 0.8852 for financial fraud, 0.8621 for online fraud, 0.8519 for cyberbullying, 0.8235 for data breach and other, and 0.8056 for women and child safety. Hacking had F1 = 0.7143 and recall 0.6667. Its overlap with neighbouring categories warrants further error analysis rather than an assumed causal explanation. Figure 6.2 retains the held-out confusion matrix.

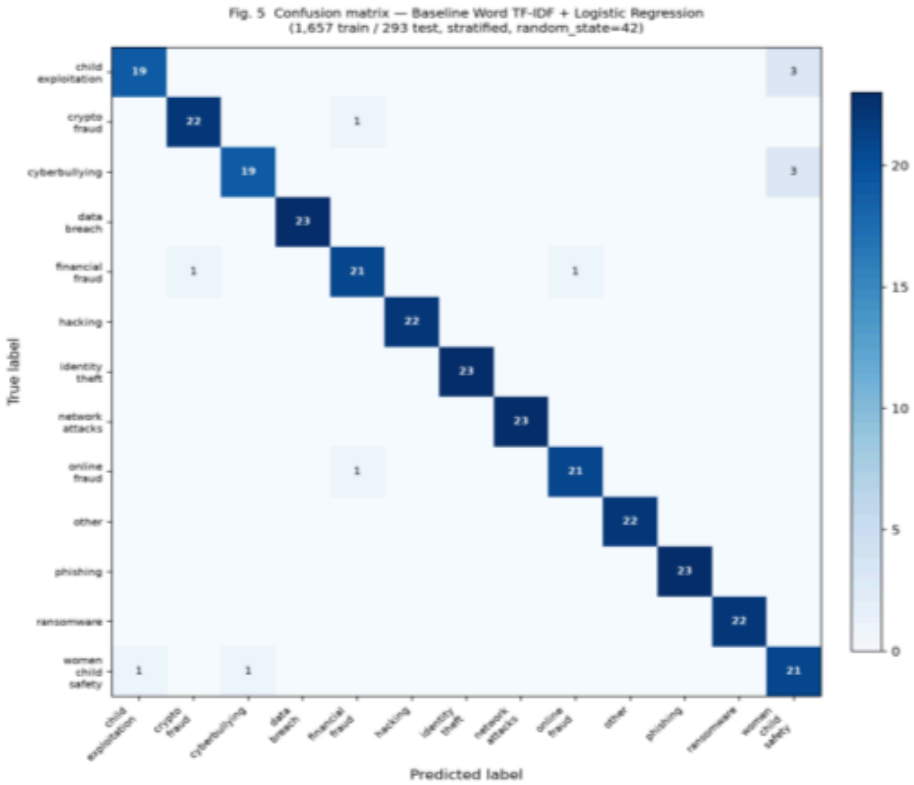


Figure 6.2 Confusion Matrix of the Baseline Model on the 390-Sample Held-Out Test Partition

The independent challenge set contains 215 items (195 in-domain + 20 benign queries). Reported raw accuracy is 98.46%, with macro-F1 0.9846; with tau = 0.20, gated accuracy is 97.44%. These accuracy percentages correspond to the 195 in-domain items, not all 215 inputs. The 20 benign inputs separately yielded a reported 0.0% out-of-distribution false-acceptance rate. The challenge set was programmatically authored and was not validated by legal desk officers. Its results are therefore a controlled engineering check, not an operational benchmark or a substitute for the group-disjoint held-out test.

6.4 Comparison with Existing Methods

Relative to the manual baseline, CyberGuard adds a preliminary category and a shared record. Relative to the store-only portal baseline, it adds masked inference, access-controlled workflow handling, and event-linked tracking. These are functional differences; the project has not measured comparative desk productivity, response time, or error rates against a deployed external platform.

Table 6.2: Comparison of Complaint Management Approaches

Evaluation Aspect	Manual Approach	Basic Online Reporting	CyberGuard
Complaint submission	Manual or direct reporting	Digital form	Web and mobile interfaces
Classification	Human categorization	Depends on the system	Word-level TF-IDF with Logistic Regression; character-based candidates evaluated
Privacy masking	Depends on procedure	Depends on implementation	Pattern-based masking
Complaint tracking	Manual follow-up	May be available	Complaint status management
Real-time communication	Separate communication channels	Depends on implementation	Supported Socket.IO features

Table 6.2 compares broad approaches, not named systems. It records the prototype's intended functions without asserting that all external reporting platforms lack them.

The comparison does not remove the need for trained reviewers. A suggested category can be wrong, a

masking pattern can miss an identifier, and a correctly rendered control can still require external authorisation. CyberGuard structures the information available for review; it does not replace that review.

6.5 Discussion

The implementation demonstrates how a constrained classifier can contribute to an application workflow. Clients collect incident information, Express coordinates processing, MongoDB retains records, and Flask supplies a preliminary category. Masking, role restrictions, escalation, and event delivery operate around that category rather than as unrelated tools.

Masking and classification must be evaluated together because token substitution changes the text representation. Removing supported identifiers may reduce opportunities to memorise incidental strings, but this report supplies no controlled ablation proving a generalisation gain. The defensible requirement is consistent preprocessing between training and inference, followed by evaluation of both privacy errors and category errors.

The results support continued prototype development under review. They do not demonstrate performance on lawful real-world submissions, comprehensive identifier removal, production security, or legally effective external actions. Before wider use, the system needs representative-data validation, calibration analysis, privacy testing, upload hardening, and an independent security assessment [16], [18], [20].

6.6 Summary of Result

The report records functional checks across submission, storage, category assignment, evidence attachment, model-input masking, role restrictions, safety escalation, tracking, and real-time updates. The screenshots document the corresponding interfaces. A reproducible test log and broader adversarial checks would be needed to establish coverage beyond those exercised paths.

For the selected configuration, 5-fold GroupKFold CV accuracy is 79.48% ± 3.21%, with Macro-F1 = 0.7507. Held-out accuracy is 88.97% (347 / 390 correct), with Macro-F1 = 0.8896. The challenge-set raw and gated in-domain accuracies are 98.46% and 97.44%. Each result retains its evaluation basis; none is a general deployment accuracy claim. Chapter 7 uses these boundaries to define the next work.

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

CyberGuard connects complaint intake, preliminary classification, model-input masking, role-controlled access, and status tracking. The web and mobile clients share an Express backend, MongoDB persistence, and a separate Flask inference service. The implementation keeps automated assistance within a reviewable complaint workflow.

The selected classifier uses word-level TF-IDF with 1–2 n-grams and at most 10,000 features, followed by Logistic Regression (C=2.0, balanced weights, lbfgs). On 1,950 balanced synthetic complaints across 13 classes and 130 scenario families, its 5-fold GroupKFold CV accuracy is 79.48% ± 3.21%, with Macro-F1 = 0.7507. The group-disjoint held-out test (N=390) yields 88.97% accuracy (347 / 390 correct) and Macro-F1 = 0.8896. A separate 215-item challenge set contains 195 in-domain complaints and 20 benign queries; reported in-domain raw and gated accuracies are 98.46% and 97.44%. The tau = 0.20 gate, masking, and escalation support review but do not guarantee correctness, anonymity, or urgency detection.

The engineering outcome is an integrated prototype, not an autonomous cybercrime authority. A handler can begin with a category suggestion, linked evidence, and a recorded status rather than an isolated narrative. Operational benefit remains to be measured on representative submissions. Consequential decisions and external legal actions remain with authorised personnel.

7.2 Limitations

The present evaluation leaves the following limitations:

Probability estimates have not been independently calibrated. The tau = 0.20 gate is a first-line filter, not a validated risk guarantee.

Training and evaluation use synthetic narratives. Performance on lawful real-world complaint submissions remains unestablished.

Class-level results vary; ambiguous, unfamiliar, short, and code-mixed narratives require further evaluation.

Regex masking misses unsupported formats and cannot establish complete removal of personal information.

Masking the narrative does not redact scanned documents, images, or other attachments.

Classifier or event-service outages affect inference and live delivery. Keyword fallback supports intake but does not preserve statistical-model quality.

Evidence handling includes storage, OCR, and candidate identifier extraction; it does not establish file authenticity, malware safety, or cryptographic custody.

Predictions, the tau = 0.20 gate, and escalation rules require human assessment, particularly for rejected or disputed outcomes.

Production load, recovery, privacy coverage, and security have not been established through

comprehensive testing.

Table V: Limitations and Required Validation

Identified Risk / Dimension	Observed Implementation Status	Required Next-Step Validation
Corpus Nature	1,950 balanced synthetic narratives (150 per class)	Validate against lawful real-world complainant submissions.
Safety Category Boundary	Confusions between child safety and bullying	Apply hierarchical two-stage classification or disambiguation rules.
Evidence Integrity	Local disk storage with relative URI links	Introduce cryptographic hash (SHA-256) chain-of-custody tracking.
Confidence Calibration	Uncalibrated Logistic Regression probabilities	Evaluate Platt scaling or isotonic regression on separate calibration data; preserve an untouched final test.
Storage Anonymization	Inference text is masked; the database retains the raw narrative	Implement database field-level encryption for sensitive complaint text.

These limitations constrain any deployment claim. Supervised prototype use does not remove the need for a lawful data-handling basis, security controls, validated access restrictions, and review by responsible personnel. The next work should be prioritised by the consequences of an error, not only by aggregate accuracy.

7.3 Future Scope

Future work should address the following validation and implementation needs:

Obtain lawful, representative complaint data and compare new models against the documented baseline using separated evaluation groups.

Evaluate regional-language and code-mixed narratives before claiming support beyond the current data distribution.

Extend identifier detection beyond regex patterns and measure both missed identifiers and false substitutions.

Add upload inspection, evidence-authenticity support, and SHA-256-based integrity records; keep these separate from OCR.

Evaluate notification reliability for disconnected clients and improve the clarity of complainant-facing status updates.

Audit authentication, authorisation, storage exposure, file access, logging, and transport security.

Run concurrent-load, service-failure, backup, and recovery tests on the intended deployment host.

Provide an auditable reviewer mechanism for confirming or correcting predicted categories.

Assess interoperability with authorised reporting portals and case registers only after technical, procedural, and legal review.

Calibrate probability estimates on separate data and evaluate the tau = 0.20 gate across false acceptance, rejection, and safety-category errors.

Curate reviewer corrections for retraining, with consent, access controls, quality checks, and versioned evaluation.

Validate the existing category, time-window, and regional dashboard summaries before extending their analytical scope.

The immediate priorities are representative-data evaluation, stronger privacy and evidence controls, and reproducible security and workflow testing. Improvements should be measured against the retained baseline rather than inferred from additional interface features.

REFERENCES

[1] A. A. S. Mahmoud, W. Shishah, and N. R. Mistry, "ReACT_OCRS: An AI-Driven Anonymous Online Reporting System Using Synergized Reasoning and Acting in Language Models," IEEE Access, vol. 13, pp. 92800–92815, 2025. doi: 10.1109/ACCESS.2025.3571526. Available: https://ieeexplore.ieee.org/document/11007115/

[2] N. Rani, D. Singh, B. Saha, and S. K. Shukla, "Automated Classification of Cybercrime Complaints using Transformer-based Language Models for Hinglish Texts," arXiv:2412.16614, 2024. Available: https://arxiv.org/abs/2412.16614

[3] A. Kamath, A. Joshi, A. Sharma, N. R. Shetty, and L. Pramiee, "Automated Threat Detection in the Dark Web: A Multi-Model NLP Approach," 2025 13th International Symposium on Digital Forensics and Security (ISDFS), 2025. doi: 10.1109/ISDFS65363.2025.11011956. Available: https://ieeexplore.ieee.org/document/11011956/

[4] Scikit-learn developers, "TfidfVectorizer," scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html (accessed 13 Nov. 2026).

[5] Scikit-learn developers, "LogisticRegression," scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html (accessed 13 Nov. 2026).

[6] Scikit-learn developers, "GroupKFold," scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupKFold.html (accessed 13 Nov. 2026).

[7] Scikit-learn developers, "classification_report," scikit-learn documentation. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html (accessed 13 Nov. 2026).

[8] Scikit-learn developers, "Pipelines and composite estimators," scikit-learn user guide. Available: https://scikit-learn.org/stable/modules/compose.html (accessed 13 Nov. 2026).

[9] React contributors, "Quick Start," React documentation. Available: https://react.dev/learn (accessed 13 Nov. 2026).

[10] Tesseract.js contributors, "Tesseract.js: Pure JavaScript OCR." Available: https://tesseract.projectnaptha.com/ (accessed 13 Nov. 2026).

[11] Express contributors, "Using middleware," Express documentation. Available: https://expressjs.com/en/guide/using-middleware.html (accessed 13 Nov. 2026).

[12] Pallets Projects, "Quickstart," Flask documentation. Available: https://flask.palletsprojects.com/en/stable/quickstart/ (accessed 13 Nov. 2026).

[13] Indian Cyber Crime Coordination Centre, Ministry of Home Affairs, Government of India, "National Cyber Crime Reporting Portal." Available: https://www.cybercrime.gov.in/home (accessed 13 Nov. 2026).

[14] MongoDB, Inc., "Introduction to Data Modeling," MongoDB Manual. Available: https://www.mongodb.com/docs/manual/core/data-modeling-introduction/ (accessed 13 Nov. 2026).

[15] A. B. Sakpere, A. V. D. M. Kayem, and T. Ndlovu, "A Usable and Secure Crime Reporting System for Technology Resource Constrained Contexts," 2015 29th International Conference on Advanced Information Networking and Applications Workshops, pp. 424–429, 2015. doi: 10.1109/WAINA.2015.974. Available: https://ieeexplore.ieee.org/document/7096212/

[16] National Institute of Standards and Technology, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1, Jan. 2023. doi: 10.6028/NIST.AI.100-1. Available: https://www.nist.gov/itl/ai-risk-management-framework

[17] Mongoose contributors, "Schemas," Mongoose documentation. Available: https://mongoosejs.com/docs/guide.html (accessed 13 Nov. 2026).

[18] Scikit-learn developers, "Probability calibration," scikit-learn user guide. Available: https://scikit-learn.org/stable/modules/calibration.html (accessed 13 Nov. 2026).

[19] Socket.IO contributors, "Introduction," Socket.IO documentation, version 4. Available: https://socket.io/docs/v4/ (accessed 13 Nov. 2026).

[20] OWASP Foundation, "File Upload Cheat Sheet," OWASP Cheat Sheet Series. Available: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html (accessed 13 Nov. 2026).

RESEARCH PUBLICATION